

Upay AI Financial Coach — Project Documentation

*Upay AI Financial Coach is a hackathon concept prototype for **DIU CPC x upay AI Hackathon / AI DEV FEST 2026**. It uses **synthetic demo data only** and is **NOT** an official production upay service.*

Table of Contents

1. [What It Is](#) (#1-what-it-is)
2. [Architecture](#) (#2-architecture)
3. [Technology Stack](#) (#3-technology-stack)
4. [Demo Users & Personas](#) (#4-demo-users--personas)
5. [Data Models](#) (#5-data-models)
6. [API Endpoints](#) (#6-api-endpoints)
7. [Key Services & Business Logic](#) (#7-key-services--business-logic)
8. [Frontend Structure](#) (#8-frontend-structure)
9. [AI Integration \(Groq\)](#) (#9-ai-integration-groq)
10. [Authentication](#) (#10-authentication)
11. [Key Workflows](#) (#11-key-workflows)
12. [Configuration](#) (#12-configuration)
13. [Important Constraints](#) (#13-important-constraints)
14. [Quick Start](#) (#14-quick-start)

1. What It Is

Upay AI Financial Coach is an intelligence and assistance layer inside a Mobile Financial Service (MFS) experience. It helps users:

- **Understand** financial behavior through AI-powered insights.
- **Plan** confidently with budget recommendations and savings goals.
- **Act** safely through guided conversational workflows.
- **Learn** financial literacy through micro-lessons.

Core Philosophy

AI explains calculated financial evidence; it does not invent balances, move money, or access PINs. Every consequential action requires explicit human confirmation. The system relies on deterministic Python calculations for all financial figures.

Key Features

Feature	Description
Money Pulse	Financial health signal with trends.
Safe-to-Spend	Disposable income analysis relative to goals.
Money Runway	Days of liquidity remaining at current spending pace.
AI Coach	Conversational interface for guidance.
Scenario Lab	"What if" simulations for financial planning.
Bangla Support	Explanations in Bengali, English, and Banglish.

2. Architecture

- **Frontend:** React + Vite (Dev: localhost:5173)
- **Backend:** FastAPI + Uvicorn (Dev: localhost:8000)
- **Intelligence:** Groq API (with Deterministic Fallback)
- **Persistence:** SQLAlchemy 2 (SQLite dev, PostgreSQL prod)

3. Technology Stack

- **Frontend:** React 18, TypeScript, TailwindCSS, Recharts, Axios.
- **Backend:** FastAPI, Pydantic v2, PyJWT, SQLAlchemy.
- **Deployment:** Vercel (Frontend & Serverless Functions).

4. Demo Users & Personas

All demo accounts use password: `password`

Email	Name	Persona	Starting Balance
<code>demo.student@upay.local</code>	Arif Rahman	Student	₹10,500
<code>demo.salary@upay.local</code>	Nadia Islam	Salaried Worker	₹22,000
<code>demo.freelancer@upay.local</code>	Samiha Noor	Freelancer	₹15,000
<code>demo.business@upay.local</code>	Rafi Ahmed	Business Owner	₹30,000

5. Data Models

Key entities include `User` , `Account` , `Transaction` , `Budget` , `SavingsGoal` , `TrustedHelper` , and `AssistantConversation` .

Note: The system maintains a state machine for `TransactionDraft` to ensure secure "Intent-to-Action" flows.

6. API Endpoints

- **Auth:** `/auth/login` , `/auth/me`
- **Intelligence:** `/intelligence/overview` , `/intelligence/runway`
- **Transactions:** `/transactions` , `/transactions/check-impact`
- **Action Flow:** `/assistant/message` , `/assistant/actions/{id}/authorize`
- **Goals/Budgets:** `/goals` , `/budgets/recommendation`

7. Key Services & Business Logic

- `analytics_service.py` : Pure Python logic for health scores, spending summaries, and forecasting.
 - `groq_service.py` : Bounded AI layer; it only translates calculated data into natural language.
 - `transaction_draft_service.py` : A robust state machine ensuring `DRAFT` → `REVIEWED` → `CONFIRMED` → `EXECUTED` steps.
-

8. Frontend Structure

The UI is modularized into:

- **Pages:** `/dashboard`, `/insights`, `/goals`, `/scenarios`, `/history`.
 - **Components:** `CoachPanel` (Chat interface), `PinConfirmationModal` (Secure action authorization), and `ExplainMetricButton`.
-

9. AI Integration (Groq)

- **Model:** `llama-3.3-70b-versatile`.
 - **Constraint:** If the AI API fails or is disabled (`AI_ENABLED=False`), the application defaults to a deterministic text generator, ensuring the service never crashes or leaves the user without an answer.
-

10. Authentication

- **JWT-based:** Uses standard HS256 tokens with a 12-hour expiry.
 - **PIN Security:** The PIN (`1234`) is **never stored in the database**. It is a temporary runtime authorization token for the simulated transaction flow.
-

11. Key Workflows

- **Send Money:** Uses "Intent-to-Action" parsing. The AI identifies the recipient and

amount, creates a draft, and the user must authorize via PIN to commit to the ledger.

- **Ask Coach:** User asks a question → Backend runs analytics → Groq formats result → User receives natural language answer.

12. Configuration

- Environment variables are managed via `.env`.
- **Crucial:** In production, `JWT_SECRET_KEY` and `DATABASE_URL` must be updated from their demo defaults.

13. Important Constraints

- **No Real Money:** All financial operations are simulations in the database.
- **No Real PINs:** PIN 1234 is for demonstration only.
- **No Off-Platform Transfers:** Limited to the app's internal ecosystem.

14. Quick Start

1. **Backend:** `cd backend && pip install -r requirements.txt && uvicorn app.main:app --reload`
2. **Frontend:** `cd frontend && npm install && npm run dev`
3. **Login:** Open `http://localhost:5173` and use any of the provided demo personas.